SafeSU User's Manual

Guillem Cabo Pitarch & Sergi Alcaide & Marc Grau Fornt

February 17, 2026

CONTENTS

1 OVERVIEW

The SafeSU (Safe Statistics Unit) is an AHB slave capable of monitoring SoC events, enforce contention control, and identifying profiling errors on run-time. Figure "1.1" shows the structure of the unit. It is composed of an ahb wrapper (*ahb_wrapper.vhd*) that maps the SystemVerilog implementation into a VHDL module that can be instanced in SELENE and De-RISC SoCs. The SystemVerilog AHB interface (*pmu_ahb.sv*) offers support for a subset of AHB requests. This module also instances the interface agnostic SafeSU (*PMU_raw.sv*). The latter is used as the generator of the statistic unit. It generates the memory map and the instances for each of the features.

The main features are:

- **Self-test:** Allows to configure the counters' inputs to a fixed value bypassing the crossbar and ignoring the inputs. This mode allows for tests of the software and the unit under known conditions.
- **Crossbar:** Allows to route any input event to any counter.
- **Counters:** Group of simple counters with settable initial values and general control register.
- **Overflow:** Detection of overflow for counters, interrupt capable with dedicated interruption vector and per counter interrupt enable.
- **MCCU** (Maximum Contention Control Unit) : Contention control measures for each core. Interruption capable after a threshold of contention is exceeded. It accepts real contention signals or estimation through weights.
- **Alternative MCCU** (Alternative Maximum Contention Control Unit) : Contention control tailoring the PQC accelerator. Unique interruption capable after any threshold of contention is exceeded. It requires specific weight and crossbar configuration so signals match required value and index.
- **RDC** (Request Duration Counters): Provides measures of the pulse length of a given input signal (watermark). It can be used to determine maximum latency and cycles of uninterrupted contentions. Each of the counters can trigger an interrupt at a user-defined threshold.

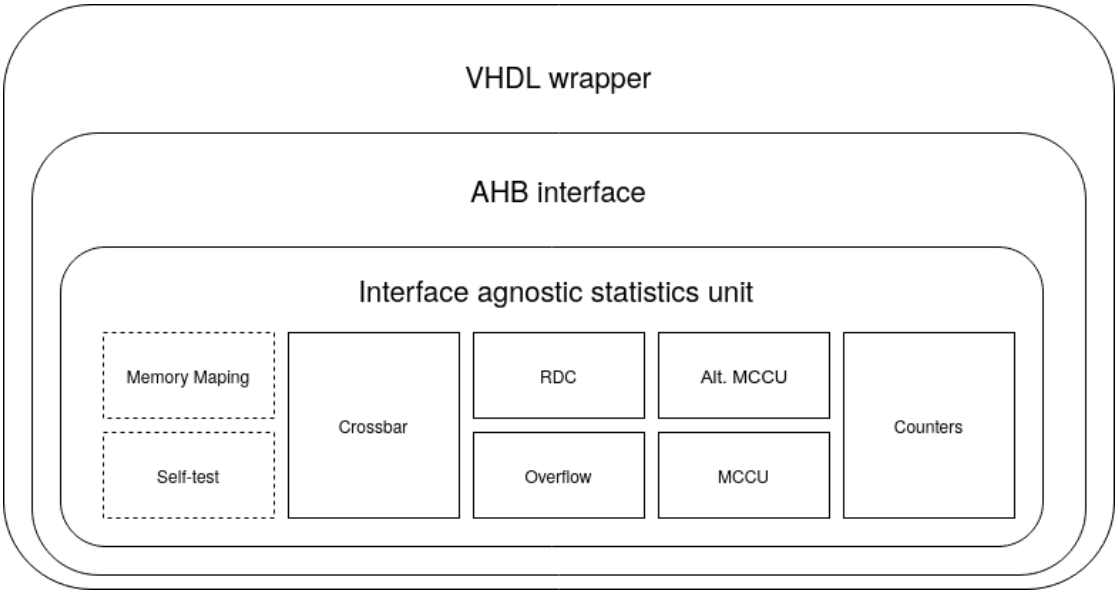


Figure 1.1: Block diagram statistics unit structure

2 OPERATION

2.1 GENERAL

The SafeSU attaches to a 128-bit wide AHB bus but only supports Single burst 32-bit accesses. AHB Lock accesses and protection control are not implemented.

The unit is compatible with GRLIB *plug&play*(P&P). Vendor and device's default configuration values are *"BSC"* and *" AHB Performance Monitoring Unit"* respectively. Regardless, release *v3.2.4* of GRMON, the unit may appear as *"Unknown device"*.

2.2 DEFAULT INPUT EVENTS

In its default implementation, the unit provides up to 256 input events. Each one of the events can be routed to any mechanism of the module through the crossbar.

Table 2.1 shows the inputs and mapping to the crossbar input for the current SELENE release. The number of signals and arrangements may change in the upcoming versions.

Table 2.1: SELENE GPL input events. 256 inputs, 6 cores

Index	Type	Source	Description
0	Debug	local	Constant HIGH, used for debug purposes or clock cycles
1	Debug	local	Constant LOW, used for debug purposes
2	Pulse	Core 0	Instruction count pipeline 0
3	Pulse	Core 0	Instruction count pipeline 1
4	Pulse	Core 0	Instruction cache miss
5	Pulse	Core 0	Instruction TLB miss
6	Pulse	Core 0	Data cache L1 miss
7	Pulse	Core 0	Data TLB miss
8	Pulse	Core 0	Branch predictor miss
9	Pulse	Core 1	Instruction count pipeline 0
10	Pulse	Core 1	Instruction count pipeline 1
11	Pulse	Core 1	Instruction cache miss
12	Pulse	Core 1	Instruction TLB miss
13	Pulse	Core 1	Data cache L1 miss
14	Pulse	Core 1	Data TLB miss
15	Pulse	Core 1	Branch predictor miss
16	Pulse	Core 2	Instruction count pipeline 0
17	Pulse	Core 2	Instruction count pipeline 1
18	Pulse	Core 2	Instruction cache miss
19	Pulse	Core 2	Instruction TLB miss
20	Pulse	Core 2	Data cache L1 miss
21	Pulse	Core 2	Data TLB miss
22	Pulse	Core 2	Branch predictor miss
23	Pulse	Core 3	Instruction count pipeline 0
24	Pulse	Core 3	Instruction count pipeline 1
25	Pulse	Core 3	Instruction cache miss
26	Pulse	Core 3	Instruction TLB miss
27	Pulse	Core 3	Data cache L1 miss
28	Pulse	Core 3	Data TLB miss
29	Pulse	Core 3	Branch predictor miss
30	Pulse	Core 4	Instruction count pipeline 0
31	Pulse	Core 4	Instruction count pipeline 1
32	Pulse	Core 4	Instruction cache miss
33	Pulse	Core 4	Instruction TLB miss
34	Pulse	Core 4	Data cache L1 miss
35	Pulse	Core 4	Data TLB miss
36	Pulse	Core 4	Branch predictor miss
37	Pulse	Core 5	Instruction count pipeline 0
38	Pulse	Core 5	Instruction count pipeline 1

Continued on next page

Table 2.1 – continued from previous page

Index	Type	Source	Description
39	Pulse	Core 5	Instruction cache miss
40	Pulse	Core 5	Instruction TLB miss
41	Pulse	Core 5	Data cache L1 miss
42	Pulse	Core 5	Data TLB miss
43	Pulse	Core 5	Branch predictor miss
44	CCS AHB	-	Agressor C1 Victim C0
45	CCS AHB	-	Agressor C2 Victim C0
46	CCS AHB	-	Agressor C3 Victim C0
47	CCS AHB	-	Agressor C4 Victim C0
48	CCS AHB	-	Agressor C5 Victim C0
49	CCS AHB	-	Agressor C0 Victim C1
50	CCS AHB	-	Agressor C2 Victim C1
51	CCS AHB	-	Agressor C3 Victim C1
52	CCS AHB	-	Agressor C4 Victim C1
53	CCS AHB	-	Agressor C5 Victim C1
54	CCS AHB	-	Agressor C0 Victim C2
55	CCS AHB	-	Agressor C1 Victim C2
56	CCS AHB	-	Agressor C3 Victim C2
57	CCS AHB	-	Agressor C4 Victim C2
58	CCS AHB	-	Agressor C5 Victim C2
59	CCS AHB	-	Agressor C0 Victim C3
60	CCS AHB	-	Agressor C1 Victim C3
61	CCS AHB	-	Agressor C2 Victim C3
62	CCS AHB	-	Agressor C4 Victim C3
63	CCS AHB	-	Agressor C5 Victim C3
64	CCS AHB	-	Agressor C0 Victim C4
65	CCS AHB	-	Agressor C1 Victim C4
66	CCS AHB	-	Agressor C2 Victim C4
67	CCS AHB	-	Agressor C3 Victim C4
68	CCS AHB	-	Agressor C5 Victim C4
69	CCS AHB	-	Agressor C0 Victim C5
70	CCS AHB	-	Agressor C1 Victim C5
71	CCS AHB	-	Agressor C2 Victim C5
72	CCS AHB	-	Agressor C3 Victim C5
73	CCS AHB	-	Agressor C4 Victim C5
74	CSS AXI	-	MLKEM0 write length – 1 beat(4-byte)
75	CSS AXI	-	MLKEM0 write length – 4 beat(16-byte)
76	CSS AXI	-	MLKEM0 read length – 1 beat(16-byte)
77	CSS AXI	-	MLKEM0 read length – 2 beat(16-byte)
78	CSS AXI	-	MLKEM0 read length – 3 beat(16-byte)

Continued on next page

Table 2.1 – continued from previous page

Index	Type	Source	Description
79	CSS AXI	-	MLKEM0 read length – 4 beat(16-byte)
80	CSS AXI	-	MLKEM0 read length – 5 beat(16-byte)
81	CSS AXI	-	MLKEM1 write length – 2 beat(16-byte)
82	CSS AXI	-	MLKEM1 write length – 4 beat(16-byte)
83	CSS AXI	-	MLKEM2 read length – 1 beat(16-byte)
84	CSS AXI	-	MLKEM2 read length – 2 beat(16-byte)
85	CSS AXI	-	MLKEM2 read length – 3 beat(16-byte)
86	CSS AXI	-	MLKEM2 read length – 4 beat(16-byte)
87	CSS AXI	-	MLKEM2 read length – 5 beat(16-byte)
88	CSS AXI	-	MLKEM3 read length – 1 beat(16-byte)
89	CSS AXI	-	MLKEM3 read length – 2 beat(16-byte)
90	CSS AXI	-	MLKEM3 read length – 3 beat(16-byte)
91	CSS AXI	-	MLKEM3 read length – 4 beat(16-byte)
92	CSS AXI	-	MLKEM3 read length – 5 beat(16-byte)
93	CSS AXI	-	MLKEM4 read length – 1 beat(16-byte)
94	CSS AXI	-	MLKEM4 read length – 2 beat(16-byte)
95	CSS AXI	-	MLKEM4 read length – 3 beat(16-byte)
96	CSS AXI	-	MLKEM4 read length – 4 beat(16-byte)
97	CSS AXI	-	MLKEM4 read length – 5 beat(16-byte)
98	CSS AXI	-	MLDSA0 write length – 1 beat(4-byte)
99	CSS AXI	-	MLDSA0 write length – 2 beat(16-byte)
100	CSS AXI	-	MLDSA0 write length – 4 beat(16-byte)
101	CSS AXI	-	MLDSA1 write length – 4 beat(16-byte)
102	CSS AXI	-	MLDSA1 read length – 1 beat(16-byte)
103	CSS AXI	-	MLDSA1 read length – 2 beat(16-byte)
104	CSS AXI	-	MLDSA1 read length – 3 beat(16-byte)
105	CSS AXI	-	MLDSA1 read length – 4 beat(16-byte)
106	CSS AXI	-	MLDSA1 read length – 5 beat(16-byte)
107	CSS AXI	-	MLDSA2 read length – 3 beat(16-byte)
108	CSS AXI	-	MLDSA2 read length – 4 beat(16-byte)
109	CSS AXI	-	MLDSA2 read length – 5 beat(16-byte)
110	CSS AXI	-	MLDSA3 read length – 1 beat(16-byte)
111	CSS AXI	-	MLDSA3 read length – 2 beat(16-byte)
112	CSS AXI	-	MLDSA3 read length – 3 beat(16-byte)
113	CSS AXI	-	MLDSA3 read length – 4 beat(16-byte)
114	CSS AXI	-	MLDSA3 read length – 5 beat(16-byte)
115	CSS AXI	-	MLDSA4 read length – 1 beat(16-byte)
116	CSS AXI	-	MLDSA4 read length – 2 beat(16-byte)
117	CSS AXI	-	MLDSA4 read length – 3 beat(16-byte)
118	CSS AXI	-	MLDSA4 read length – 4 beat(16-byte)

Continued on next page

Table 2.1 – continued from previous page

Index	Type	Source	Description
119	CSS AXI	-	MLDSA4 read length – 5 beat(16-byte)
120	-	-	Filler signal, constant 0
...	-	-	Filler signal, constant 0
241	-	-	Filler signal, constant 0
242	CCS AHB	-	Aggressor others Victim C0
243	-	-	Filler signal, constant 0
244	-	-	Filler signal, constant 0
245	-	-	Filler signal, constant 0
246	-	-	Filler signal, constant 0
247	-	-	Filler signal, constant 0
248	CCS AHB	-	Generic, Aggressor 0 Victim 0
249	CCS AHB	-	Generic, Aggressor 1 Victim 0
250	CCS AHB	-	Generic, Aggressor 2 Victim 0
251	CCS AHB	-	Generic, Aggressor 3 Victim 0
252	CCS AHB	-	Generic, Aggressor 4 Victim 0
253	CCS AHB	-	Generic, Aggressor 5 Victim 0
254	CCS AHB	-	Residual, Victim 0
255	-	-	Filler signal, constant 0

Table 2.2: Mapping from Master qos (MQ) to IPs on GPL release.

MQ	Bus of origin	Bus ID	Description
0	AHB	0	NOEL-V CORE
1	AHB	1	NOEL-V CORE
2	AHB	2	NOEL-V CORE
3	AHB	3	NOEL-V CORE
4	AHB	4	NOEL-V CORE
5	AHB	5	NOEL-V CORE
6	AHB	6	GR Ethernet MAC
7	AHB	7	GRDMAC2 DMA Controller
8	AHB	8	AHB Debug UART
9	AHB	9	JTAG Debug Link
10	AHB	10	EDCL master interface
11	-	-	Unused
12	-	-	Unused
13	-	-	Unused
14	-	-	Unused

Table 2.3: Maping from Master qos (MQ) to IPs on non-GPL release

MQ	Bus of origin	Bus ID	Description
0	AHB	0	NOEL-V CORE
1	AHB	1	NOEL-V CORE
2	AHB	2	NOEL-V CORE
3	AHB	3	NOEL-V CORE
4	AHB	4	NOEL-V CORE
5	AHB	5	NOEL-V CORE
6	AHB	6	GR Ethernet MAC
7	AHB	7	GRSPW2 SpaceWire
8	AHB	8	GRSPW2 SpaceWire
9	AHB	9	CAN-FD Controller with DMA
10	AHB	10	CAN-FD Controller with DMA
11	AHB	11	GRDMAC2 DMA Controller
12	AHB	12	AHB Debug UART
13	AHB	13	JTAG Debug Link
14	AHB	14	EDCL master interface

Signals labeled as *debug* are fix inputs that can be used to test hardware or software with known inputs. **Event 0** (fix '1') can be used to measure the **number of elapsed cycles**.

Signals of type *CCS* can be used to compute the total contention cycle stack of the system. These signals become high at the first rising edge of the clock after a given condition or event has been detected. They remain active until the condition or event that they are measuring becomes low. This behavior allows measuring the length of clock cycles. When generating CCS signals, the user must consider if they want to allow back to back events and generate the input signals accordingly at RTL level.

The generic and residual contention metrics have the following meanings:

- **Generic contention**: The previous request has been served, but has left the slave in an unready state, making this core's request be delayed this cycle. We assume that the slave would be ready if the previous master had not accessed it.
- **Residual contention**: We observed that, when using the gaisler L2-lite cache, if a htrans "10"(NONSEQ) happens after a non-idle transaction, the response to the current master (the one that issued the "10") transaction is delayed by one cycle. THIS IS GAISLER L2 LITE specific, thus, not portable

2.3 Main configuration and self-test

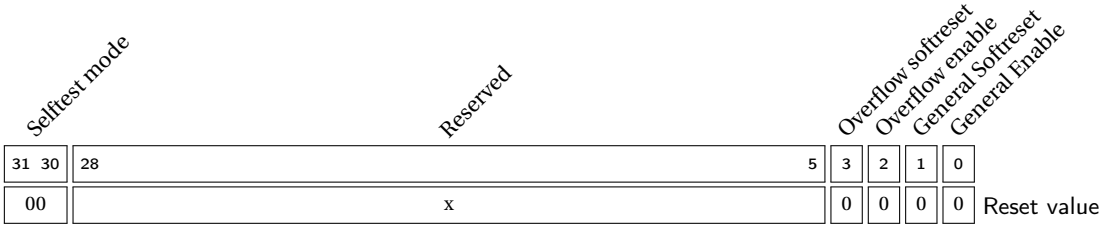
Reset and enable of overflow and regular counters can be performed with register 2.1. All signals are active high.
Self-test mode allows to bypass the input events from the crossbar and instead use a specific

input pattern where signals are constant. This mode can be used for debugging. After the addition of the crossbar and debug inputs, there is a certain overlap. The same results can be achieved with the correct crossbar configuration. Nevertheless, it has been included in this release for compatibility.

These are the Self-test modes for each configuration value of the fied *selftest mode* in register 2.1:

- 0b00: Events depend on the crossbar. Self-test is disabled
- 0b01: All signals are set to 1.
- 0b10: All signals are set to 0.
- 0b11: Signal 0 is set to 1. The remaining signals are set to 0.

Register 2.1: BASE CONFIGURATION REGISTER (0x000)



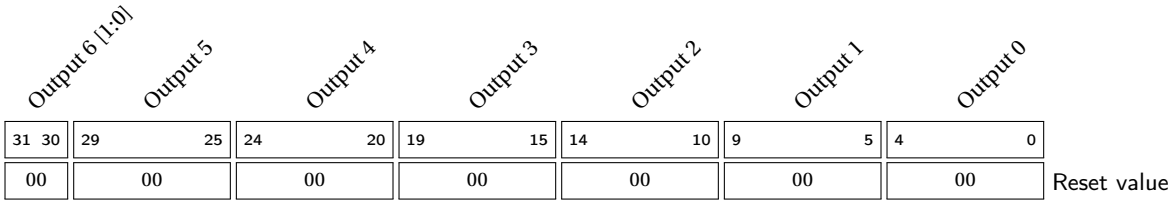
2.4 CROSSBAR

This feature allows routing any of the input signals of table 2.1 or 2.2 into any of the 24 counters of the SafeSU. Each one of the counters has a 5-bit configuration value for the 32 input version of the crossbar, 7-bit for the 128 version and 8-bit for the 256 version. For simplicity this section describes the 32 entry variant of the crossabar. Consider that the width of the configuration field is $log2(N\_inputs)$ and the total amount of configuration registers $log2(N\_inputs) * N\_inputs$.
Configuration values are stored in registers 2.2, 2.3, 2.4,2.5. All the configuration values are consecutive. Thus some values may have configuration bits in two consecutive memory addresses. Examples of this are Output 6, 12, 19 in our current configuration. As a consequence, the previous outputs may require two writes to configure the desired input signal.

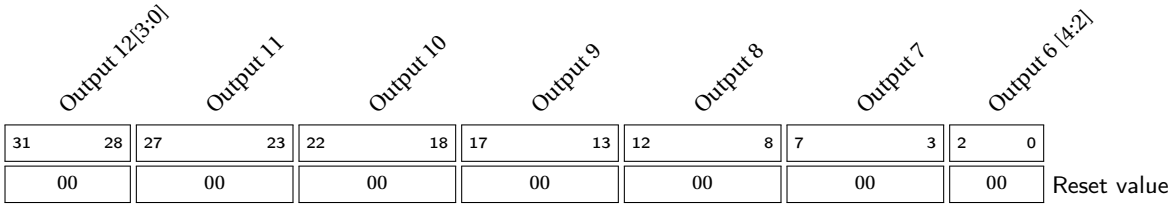
Configuration fields match one to one with the internal counters. So the field *output 0* matches with *counter 0*, *output 1* with *counter 1* and so on.

As a usage example, suppose the user wants to route the signal *pmu_events(0).icnt(0)* to the internal *counter 0*. The field *Output 0* of register 2.2 shall match the Index of the signal in table 2.1. In this case, the index is 2. After this configuration, the event count will be recorded in *counter 0*. The addresses for counter values are indicated in figure 2.1.

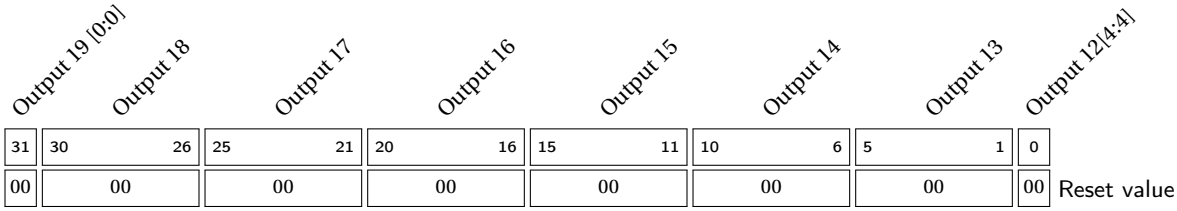
Register 2.2: CROSSBAR CONFIGURATION REGISTER 0 (0x0AC)



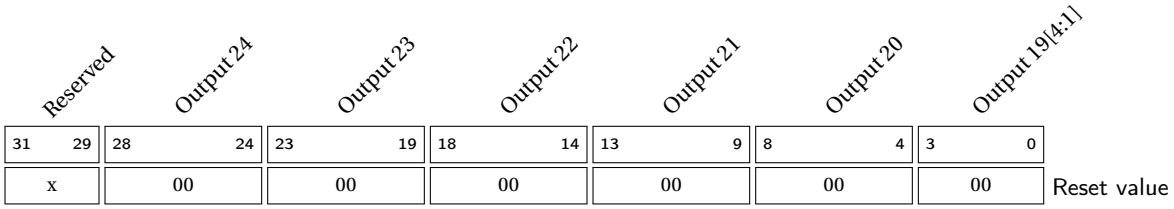
Register 2.3: CROSSBAR CONFIGURATION REGISTER 1 (0x0B0)



Register 2.4: CROSSBAR CONFIGURATION REGISTER 2 (0x0B4)



Register 2.5: CROSSBAR CONFIGURATION REGISTER 3 (0x0B8)



Signal routing is important since some of the SafeSU features are only available at different crossbar outputs. Table 2.4 shows the available capabilities for each one of the outputs.

[0]RDC and MCCU signals are only available if the assigned core is synthesized.

Table 2.4: Crossbar outputs and SafeSU capabilities

Output	Counters	Overflow	MCCU	RDC	ALT MCCU
0	Yes	Yes	Core 0	Yes	No
1	Yes	Yes	Core 0	Yes	No
2	Yes	Yes	Core 1	Yes	No
3	Yes	Yes	Core 1	Yes	No
4	Yes	Yes	Core 2	Yes	No
5	Yes	Yes	Core 2	Yes	No
6	Yes	Yes	Core 3	Yes	No
7	Yes	Yes	Core 3	Yes	No
8	Yes	Yes	Core 4	Yes	Yes
9	Yes	Yes	Core 4	Yes	Yes
10	Yes	Yes	Core 5	Yes	Yes
11	Yes	Yes	Core 5	Yes	Yes
12	Yes	Yes	No	No	Yes
13	Yes	Yes	No	No	Yes
14	Yes	Yes	No	No	Yes
15	Yes	Yes	No	No	Yes
16	Yes	Yes	No	No	Yes
17	Yes	Yes	No	No	Yes
18	Yes	Yes	No	No	Yes
19	Yes	Yes	No	No	Yes
20	Yes	Yes	No	No	Yes
21	Yes	Yes	No	No	Yes
22	Yes	Yes	No	No	Yes
23	Yes	Yes	No	No	Yes

2.5 COUNTERS

The unit in the default configuration contains 24 counters, 32-bit each. Figures 2.1 and 2.2 indicate the memory address where each counter's value can be access. Counter values can be **read** or **written**, thus allowing to set the initial value of the counters.

Enable and reset is managed by the base configuration register 2.1.

Counters can overflow. In such a case, the count will wrap back to 0 and keep counting. Section 2.6 describes how to enable the overflow detection interrupts.

Figure 2.1: 0 to 12 Counter adresses

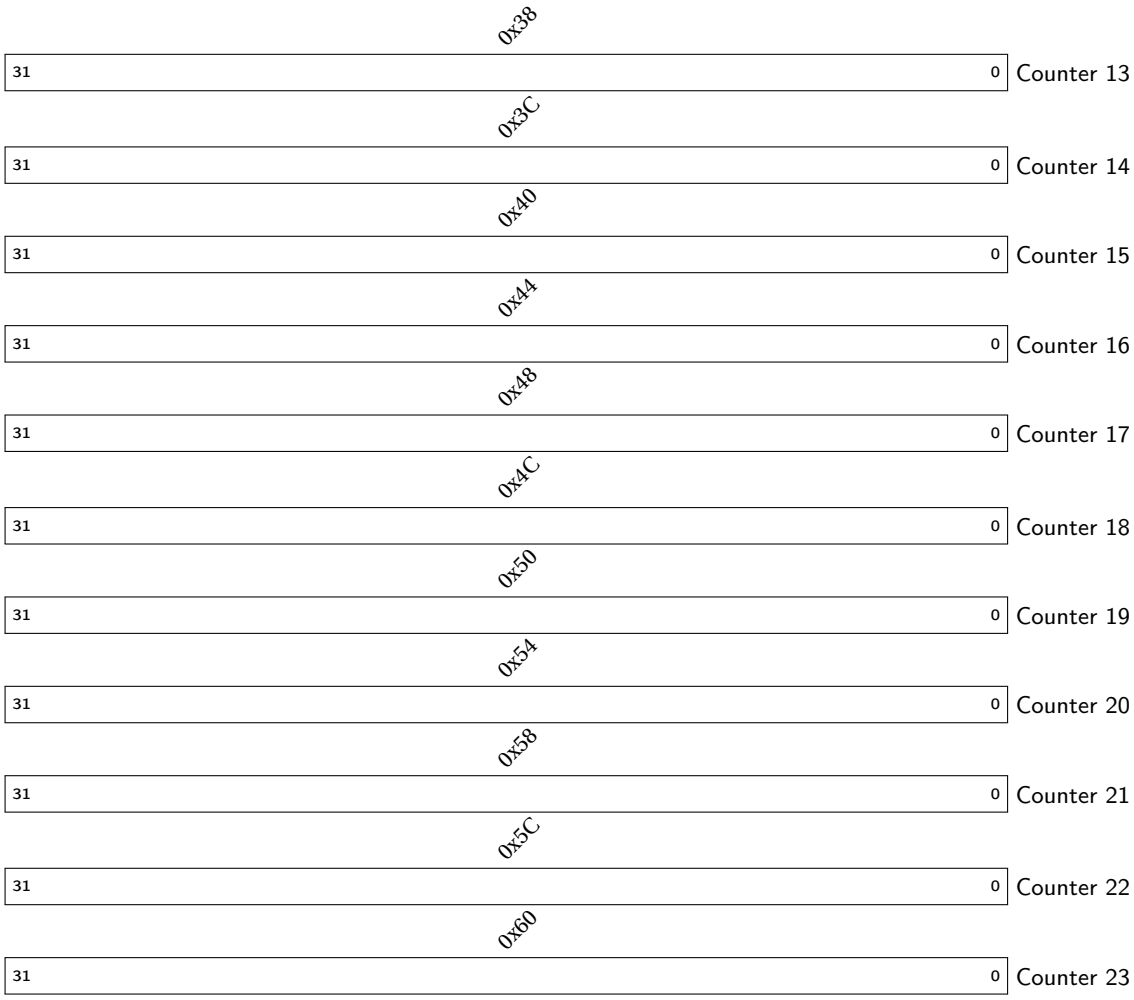


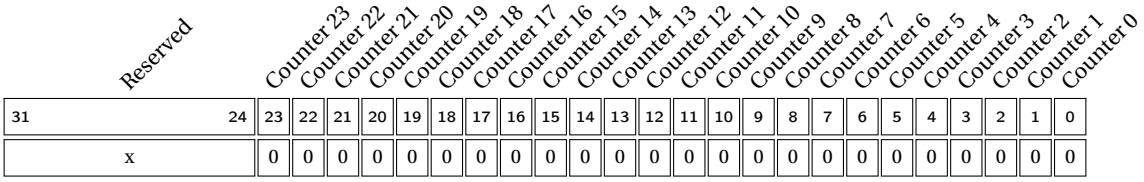
Figure 2.2: 12 to 24 Counter addresses

2.6 OVERFLOW

The user can enable overflow detection for each one of the counters in section 2.5. Enables are active high and individual for each counter, as indicated in register 2.6. If a counter with overflow detection active wraps over the maximum value, the corresponding bit of register 2.7 will become 1, and AHB interrupt number 6 will become active.

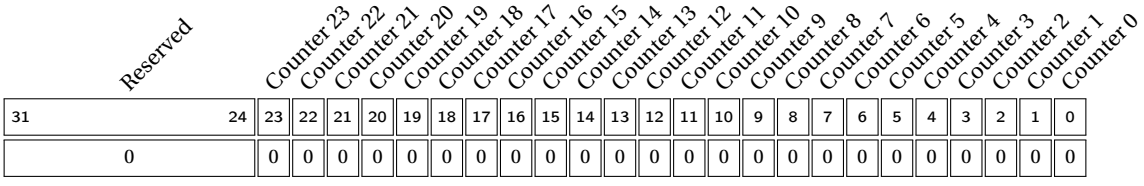
The default AHB interrupt mapping can be modified within the file *ahb_wrapper.vhd*.

Register 2.6: OVERFLOW INTERRUPT ENABLE MASK (0x064)



	Reserved	Counter 23	Counter 22	Counter 21	Counter 20	Counter 19	Counter 18	Counter 17	Counter 16	Counter 15	Counter 14	Counter 13	Counter 12	Counter 11	Counter 10	Counter 9	Counter 8	Counter 7	Counter 6	Counter 5	Counter 4	Counter 3	Counter 2	Counter 1	Counter 0
31 24 / 23	31 24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reset value	x	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Register 2.7: OVERFLOW INTERRUPT VECTOR (0x068)



	Reserved	Counter 23	Counter 22	Counter 21	Counter 20	Counter 19	Counter 18	Counter 17	Counter 16	Counter 15	Counter 14	Counter 13	Counter 12	Counter 11	Counter 10	Counter 9	Counter 8	Counter 7	Counter 6	Counter 5	Counter 4	Counter 3	Counter 2	Counter 1	Counter 0
31 24 / 23	31 24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

2.7 MCCU

The Maximum-Contention Control Unit (MCCU) allows to monitor a subset of the input events and track the approximate contention that they will cause. Currently, events assigned to counters 0 to 11 can be used as inputs of the MCCU. Thanks to the crossbar, any of the SoC signals can be used by the MCCU. In this section we use the 4 core version as a reference. In all configurations one quota register is assigned to each core and one weight is asigned to each signal.

Register 2.8: MCCU MAIN CONFIGURATION FOR 6 CORE CONFIGURATIONS (0x074)

Enable Hardware Quota	Reserved	Soft reset RDC	Enable RDC	Update Quota PQC Write	Update Quota PQC Read	Update Quota PQC Read & Write	Update Quota Core 5	Update Quota Core 4	Update Quota Core 3	Update Quota Core 2	Update Quota Core 1	Update Quota Core 0	Soft reset MCCU	Enable reset MCCU
31	30 … 13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	x	0	0	0	0	0	0	0	0	0	0	0	0	0

Reset value

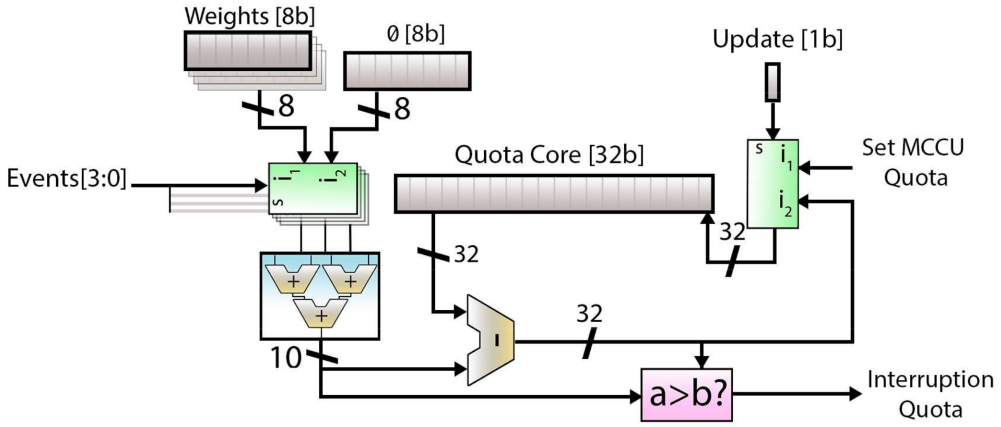


Figure 2.3: Block diagram MCCU mechanism for one core.

Figure 2.3 shows the internal elements required to monitor the quota consumption of one core, given four input events. When events are active, they pass the value assigned in the weight register 2.9 for the given signal to a series of adders. The addition is subtracted from the corresponding quota register 2.5. When the remaining quota is smaller than the cycle contention, an interrupt is triggered.

When the "Enable Hardware Quota" bit is toggled, this interrupt is routed to the AHB controller directly to block contending core access to the shared bus. Thus, disabling the interrupt lines and performing the decisions without software intervention.

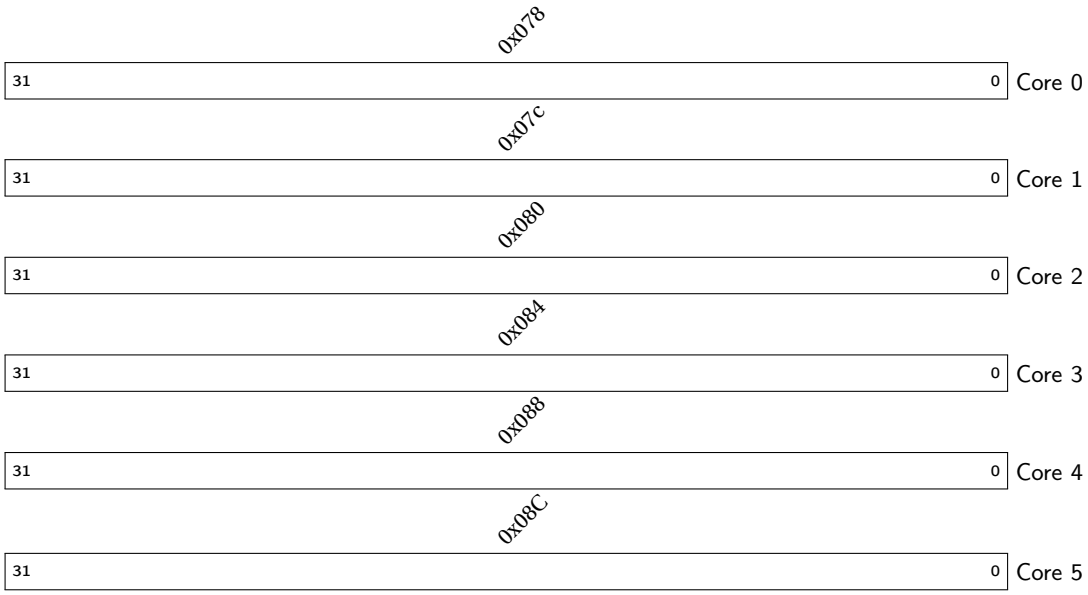


Figure 2.4: MCCU Quota limits for each core

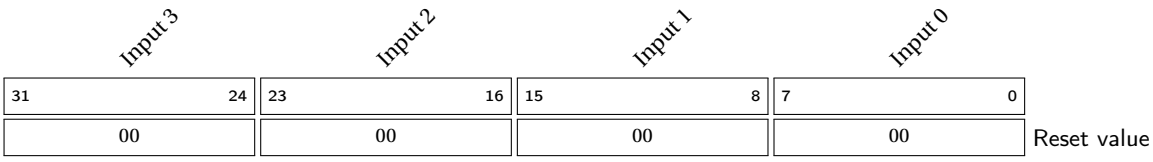
Figure 2.5: MCCU Current remaining Quota for each core

In the current release, the MCCU can be reset and activated with the respective fields of register 2.8. The fields labeled as *Update Quota core* are used to update the available quota of each core (figure 2.5). While *Update Quota core* is high, the content is assigned to the avail-

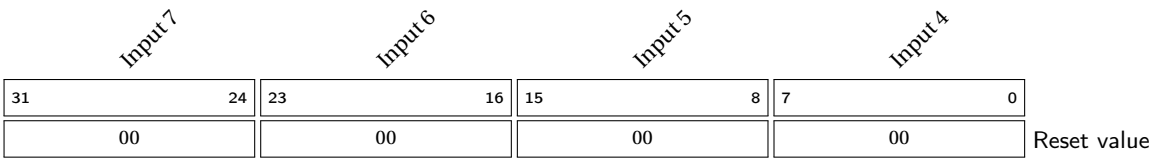
able quota. Once released (low), the available quota can start to decrease if the MCCU is active. The current quota can be read while the unit is active.

In the current release, each core can monitor two input events. The MCCU module is parametric and more events can be provided in future releases. Table 2.4 shows the features available for each crossbar output. Under the column MCCU, you can see towards which core quota the event will be computed. The unit provides one interruption for each of the monitored cores.
Weights for each monitored event are registered in registers 2.9 and 2.10. Currently, each weight is an 8-bit field. Each input of the MCCU maps directly to the outputs of the crossbar. Thus the weight for the MCCU input 0 corresponds to the signal in crossbar output 0.

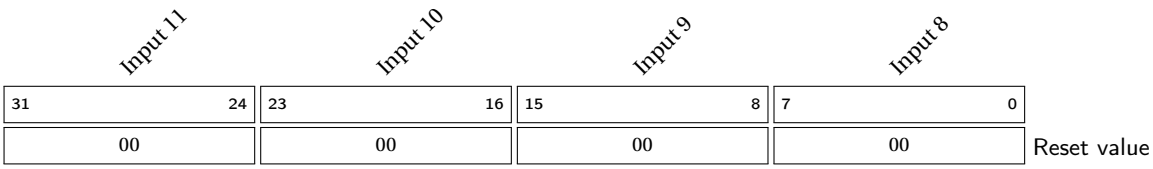
Register 2.9: MCCU EVENT WEIGHTS REGISTER 0 (SHARED WITH RDC) (0x0C0)



Register 2.10: MCCU EVENT WEIGHTS REGISTER 1 (SHARED WITH RDC) (0x0C4)



Register 2.11: MCCU EVENT WEIGHTS REGISTER 2 (SHARED WITH RDC) (0x0C8)



2.8 ALTERNATIVE MCCU

PQC accelerator monitoring is supported by the SafeSU using a secondary MCCU.
This implementation differs from the original MCCU in that the input event signals from the crossbar, and weights in a particular interconnection, are shared among 3 additional PQC cores, Read & Write, Read, and Write cores.

The event signals must be arranged from the crossbar output following a specific order (indexed by the output Crossbar ID) specified by the alternative MCCU hardware implementation. These are summarized by the particular PQC benchmark configurations, including MLKEM encapsulation (table 2.5), MLKEM decapsulation (table 2.6), MLDSA signature (table 2.7), and MLDSA verification (table 2.8).

Table 2.5: Counter Assignment for MLKEM Encapsulation.

Event ID	**Crossbar ID**	**Description**
74	8	MLKEM0 write length – 1 beat(4-byte)
81	9	MLKEM1 write length – 2 beat(16-byte)
75	10	MLKEM0 write length – 4 beat(16-byte)
83	11	MLKEM2 read length – 1 beat(16-byte)
84	12	MLKEM2 read length – 2 beat(16-byte)
85	13	MLKEM2 read length – 3 beat(16-byte)
86	14	MLKEM2 read length – 4 beat(16-byte)
87	15	MLKEM2 read length – 5 beat(16-byte)
88	16	MLKEM3 read length – 1 beat(16-byte)
89	17	MLKEM3 read length – 2 beat(16-byte)
90	18	MLKEM3 read length – 3 beat(16-byte)
91	19	MLKEM3 read length – 4 beat(16-byte)
92	20	MLKEM3 read length – 5 beat(16-byte)
95	21	MLKEM4 read length – 3 beat(16-byte)
96	22	MLKEM4 read length – 4 beat(16-byte)
97	23	MLKEM4 read length – 5 beat(16-byte)

Table 2.6: Counter Assignment for MLKEM Decapsulation.

Event ID	Crossbar ID	Description
74	8	MLKEM0 write length – 1 beat(4-byte)
81	9	MLKEM1 write length – 2 beat(16-byte)
82	10	MLKEM1 write length – 4 beat(16-byte)
76	11	MLKEM0 read length – 1 beat(16-byte)
77	12	MLKEM0 read length – 2 beat(16-byte)
78	13	MLKEM0 read length – 3 beat(16-byte)
79	14	MLKEM0 read length – 4 beat(16-byte)
80	15	MLKEM0 read length – 5 beat(16-byte)
93	16	MLKEM4 read length – 1 beat(16-byte)
94	17	MLKEM4 read length – 2 beat(16-byte)
95	18	MLKEM4 read length – 3 beat(16-byte)
96	19	MLKEM4 read length – 4 beat(16-byte)
97	20	MLKEM4 read length – 5 beat(16-byte)
90	21	MLKEM3 read length – 3 beat(16-byte)
91	22	MLKEM3 read length – 4 beat(16-byte)
92	23	MLKEM3 read length – 5 beat(16-byte)

Table 2.7: Counter Assignment for MLDSA Signature.

Event ID	Crossbar ID	Description
98	8	MLDSA0 write length – 1 beat(4-byte)
99	9	MLDSA0 write length – 2 beat(16-byte)
101	10	MLDSA1 write length – 4 beat(16-byte)
115	11	MLDSA4 read length – 1 beat(16-byte)
116	12	MLDSA4 read length – 2 beat(16-byte)
117	13	MLDSA4 read length – 3 beat(16-byte)
118	14	MLDSA4 read length – 4 beat(16-byte)
119	15	MLDSA4 read length – 5 beat(16-byte)
110	16	MLDSA3 read length – 1 beat(16-byte)
111	17	MLDSA3 read length – 2 beat(16-byte)
112	18	MLDSA3 read length – 3 beat(16-byte)
113	19	MLDSA3 read length – 4 beat(16-byte)
114	20	MLDSA3 read length – 5 beat(16-byte)
107	21	MLDSA2 read length – 3 beat(16-byte)
108	22	MLDSA2 read length – 4 beat(16-byte)
109	23	MLDSA2 read length – 5 beat(16-byte)

Table 2.8: Counter Assignment for MLDSA Verification.

Event ID	Crossbar ID	Description
98	8	MLDSA0 write length – 1 beat(4-byte)
99	9	MLDSA0 write length – 2 beat(16-byte)
100	10	MLDSA0 write length – 4 beat(16-byte)
102	11	MLDSA1 read length – 1 beat(16-byte)
103	12	MLDSA1 read length – 2 beat(16-byte)
104	13	MLDSA1 read length – 3 beat(16-byte)
105	14	MLDSA1 read length – 4 beat(16-byte)
106	15	MLDSA1 read length – 5 beat(16-byte)
110	16	MLDSA3 read length – 1 beat(16-byte)
111	17	MLDSA3 read length – 2 beat(16-byte)
112	18	MLDSA3 read length – 3 beat(16-byte)
113	19	MLDSA3 read length – 4 beat(16-byte)
114	20	MLDSA3 read length – 5 beat(16-byte)
107	21	MLDSA2 read length – 3 beat(16-byte)
108	22	MLDSA2 read length – 4 beat(16-byte)
109	23	MLDSA2 read length – 5 beat(16-byte)

As consequence of using crossbar output signals [8]-[11], overlapping MCCU/RDC cores [4]-[5] are not available during PQC monitoring with the alternative MCCU.
The remaining crossbar outputs [0]-[7] can be used either by the MCCU/RDC cores [0]-[3] or counters [0]-[7]. Table 2.9 shows the default configuration where counters [0]-[1] are used for debug purposes while counters [2]-[6] are assigned with the events representing core [0] contention from other cores [1]-[5].

Table 2.9: Counter Assignment for MLDSA Verification.

Crossbar ID	Type	Source	Description
0	Debug	local	Constant HIGH, used for debug purposes or clock cycles
1	Debug	local	Constant LOW, used for debug purposes
2	CCS AHB	-	Aggressor C1 Victim C0
3	CCS AHB	-	Aggressor C2 Victim C0
4	CCS AHB	-	Aggressor C3 Victim C0
5	CCS AHB	-	Aggressor C4 Victim C0
6	CCS AHB	-	Aggressor C5 Victim C0
7	-	-	Filler signal, constant 0

The 3 extended cores are highly integrated with the original MCCU, which quota limits (registers from figure 2.6) and remaining quota (registers from figure 2.7) are accessed as any other MCCU register. Such PQC core quotas are updated using the same configuration register 2.8, by asserting bits [8]-[10] to high, one for each PQC core.

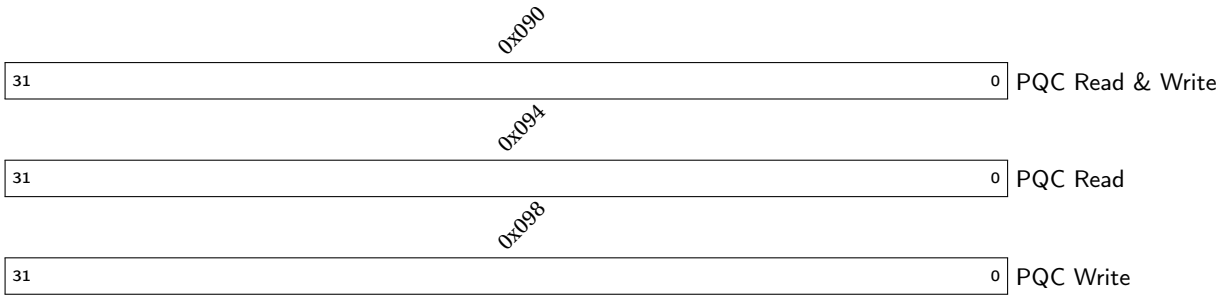


Figure 2.6: Alternative MCCU Quota limits for each PQC core

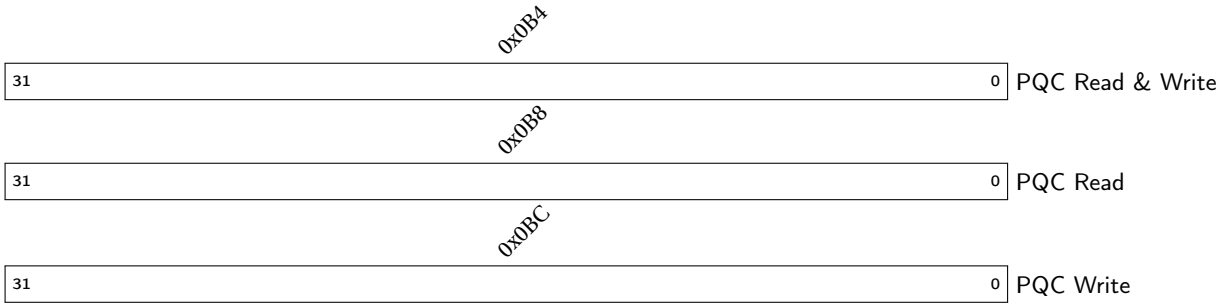


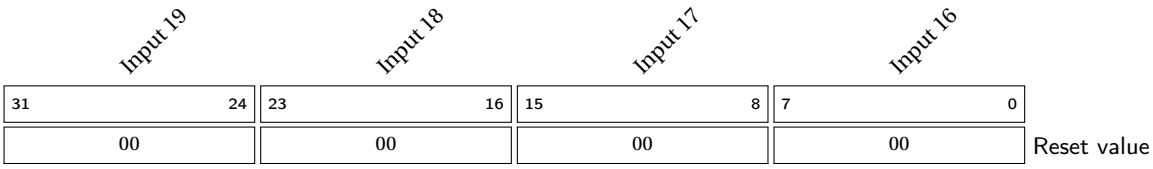
Figure 2.7: Alternative MCCU Current remaining Quota for each PQC core

In term of the weights, the alternative MCCU also does differ from the original MCCU, sharing, duplicating and nullifying the weights instead of having one weight for each event signal, while implementing weight registers Inputs [12-19] through registers 2.12 and 2.13.

Register 2.12: ALTERNATIVE MCCU EVENT WEIGHTS REGISTER 0 (NOT SHARED WITH RDC) (0x0CC)

Input 15		Input 14		Input 13		Input 12		
31	24	23	16	15	8	7	0	
00		00		00		00		Reset value

Register 2.13: ALTERNATIVE MCCU EVENT WEIGHTS REGISTER 1 (NOT SHARED WITH RDC) (0x0D0)



The event signals routing from the crossbar output is fixed so the custom weight wiring applies the following functions.

$$PQCRead\&Write[0] = PQCRead + PQCWrite$$

$$PQCRead[1] = 16 \cdot (\sum_{i=11}^{15} (i-10) \cdot \text{event}[i] + \sum_{i=16}^{20} (i-15) \cdot \text{event}[i] + \sum_{i=21}^{23} (i-18) \cdot \text{event}[i])$$

$$PQCWrite[2] = 4 \cdot \text{event}[8] + 32 \cdot \text{event}[9] + 64 \cdot \text{event}[10]$$

Analyzing the functions the number, values and connections of weights with the crossbar output events can be generated, resulting in table 2.10. As the table shows, only weight inputs [12]-[17] are actually used, and these should be configured with the values shown so the weights apply the correct multiplication factor on each crossbar output, resulting in the functions from above.

Table 2.10: Alternative MCCU weights for correct function application.

Weight input ID	**Value**
12	4
13	16
14	32
15	48
16	64
17	80
18	-
19	-

The custom wiring is what links input events and the required weight, or lack of, for the alternative MCCU to work properly for the PQC requirements. Translation to another application, module or platform may require rewiring and computing a new set of weights tailoring different functions.

2.9 RDC

The Request Duration Counter or RDC is comprised of a set of 8-bit counters and comparators that allow monitoring the length of a CCS signal, record the number of clock cycles of the

longest pulse and compare such value with the defined weight.

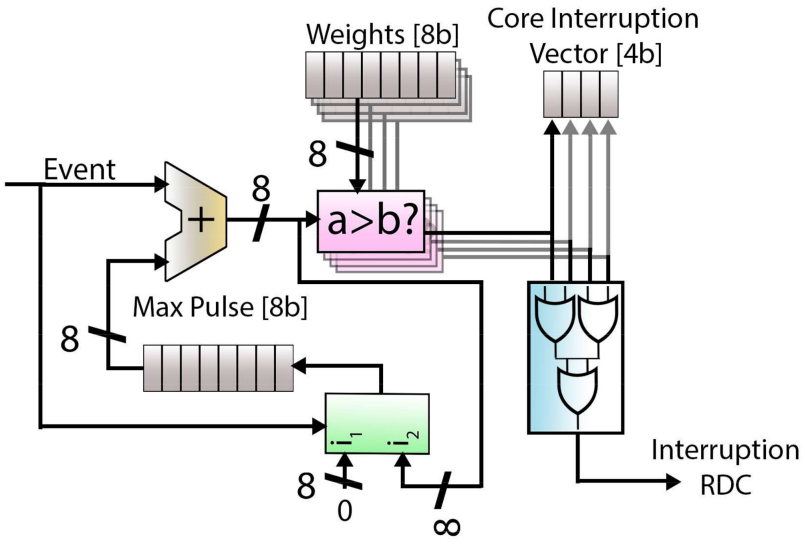


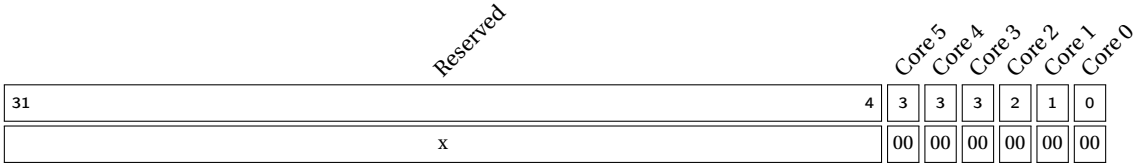
Figure 2.8: Block diagram RDC mechanism.

The current release provides monitoring for crossbar outputs 0 to 11 for the hexa-core. Note that the amount of registers changes with the amount of cores, and figures and addresses are representative of the 4 core configuration. The weights for each signal are shared with the MCCU and are stored in registers 2.15, 2.16 and 2.17. Weights are 8-bit fields.Counters have overflow protection, preventing the count from wrapping over the maximum value. The maximum value for each event (watermarks), are stored in registers 2.18, 2.19 and 2.20.

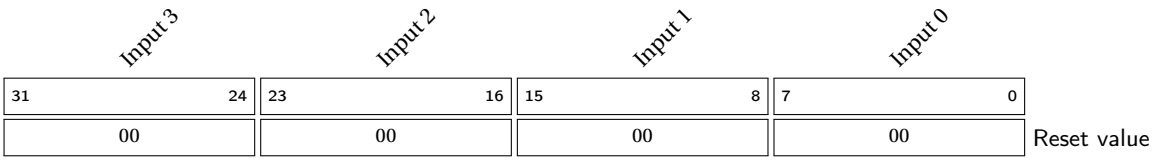
The RDC shares the main configuration register with the MCCU (register 2.8). Through this register, the unit can be reset and enabled through the corresponding fields. Such fields are active high signals.

The unit does provide access to the internal interrupt vector (register 2.14), but such information is redundant and may be removed in future releases. Given the current watermarks and assigned weights, the events responsible for the interrupt can be identified.
Weights not configured or with value 0 will not raise interruptions.

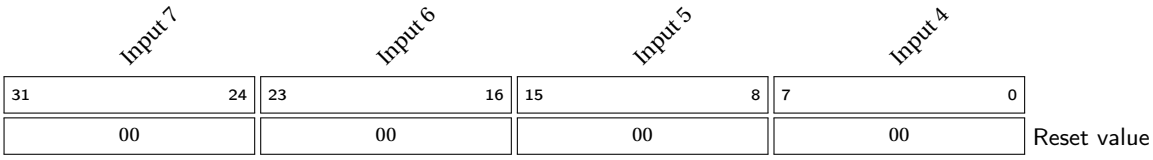
Register 2.14: RDC INTERRUPT VECTOR (0x0D4)



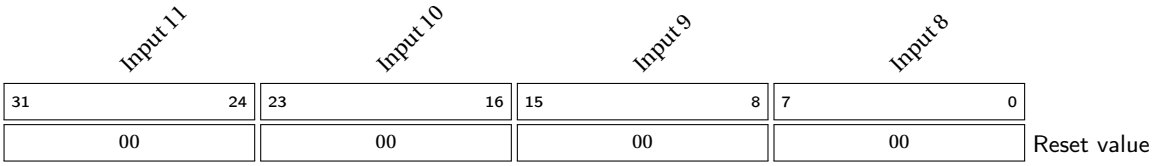
Register 2.15: RDC EVENT WEIGHTS REGISTER 0 (SHARED WITH MCCU) (0x0C0)



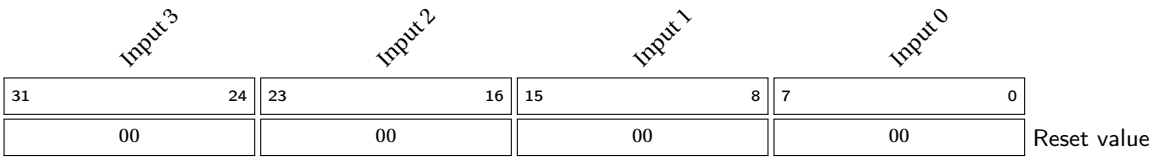
Register 2.16: RDC EVENT WEIGHTS REGISTER 1 (SHARED WITH MCCU) (0x0C4)



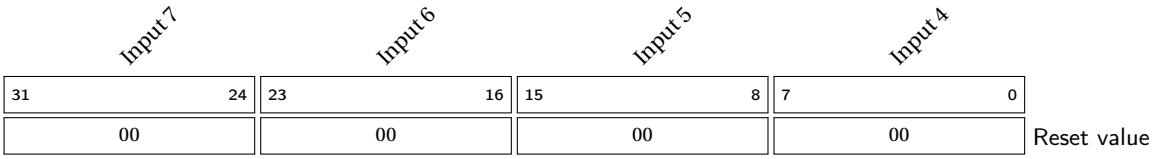
Register 2.17: RDC EVENT WEIGHTS REGISTER 2 (SHARED WITH MCCU) (0x0C8)



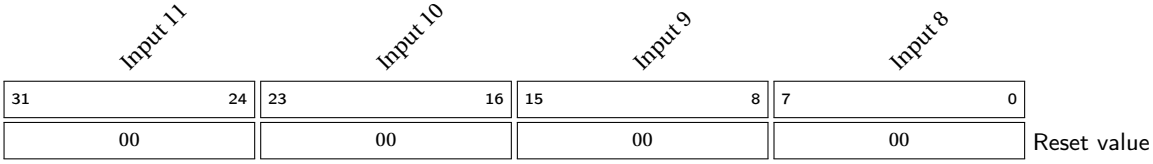
Register 2.18: RDC WATERMARK REGISTER 0 (0x0D8)



Register 2.19: RDC WATERMARK REGISTER 1 (0x0DC)



Register 2.20: RDC WATERMARK REGISTER 2 (0x0E0)



2.10 SOFTWARE SUPPORT

The unit can be configured by the user at a low level following the description of previous sections and the documents associated for each unit. In addition we provide a small bare-metal driver under the *drivers* directory of the SafeSU repository.

Currently 3 configurations are provided. *4-core* and *6-core* folders have the driver for default configurations. *6-core-256e* folder has a driver for an hexacore with 256 entries on the crossbar.

The drivers are composed of three files.

- *pmu_vars.h* : Defines a set of constants with the RTL parameters that generated the unit. Such constants allow the reusing of functions among hardware configurations.
- *pmu_hw.h*: Defines the memory position of the SafeSU within the memory map of the SoC. It also contains the prototypes of each function of the driver.
- *pmu_hw.c*: Contains the definition of each of the functions.

Given the current development status, the driver is not autogenerated, and **modifications to the RTL may require manual changes on the previous files**.

3 CONFIGURATION OPTIONS

Table 3.1 shows the configuration parameters exposed by *ahb_wrapper.vhd*. Given the current development status, changes to the configuration options may require manual modifications to internal modules and software drivers. Future releases will expose parameters to enable individual SafeSU features and allow for more flexibility.

Table 3.1: Configuration options (VHDL ports)

Generic	Function	Allowed range	Default
HADDR	AHB base address	0 to 16#fff#	0
HMASK	AHB address mask	0 to 16#fff#	16#fff#
N_REGS	Total of accessible registers	2 to 64	43
SafeSU_COUNTERS	Number of generic counters. Same as crossbar outputs	1 to 32	24
N_SOC_EV	SoC signals. Inputs to the crossbar	1 to 64	32
REG_WIDTH	Size of registers and counters	32 **or** 64	32

4 SIGNAL DESCRIPTIONS

Table 4.1 shows the interface of the core (VHDL ports).

Table 4.1: Signal descriptions (VHDL ports)

Signal name	Field	Type	Function	Active
RST		Input	Reset	Low
CLK		Input	AHB master bus clock	-
SafeSU_EVENTS		Input	Input for regular SoC events	-
CCS_CONTENTION		Input	Input for contention cycle stack signals that measure contention	-
CCS_LATENCY		Input	Input for contention cycle stack signals that measure access latency	-
AHBSI	*	Input	AHB slave input signals	-
AHBSO	*	Output	AHB slave output signals, includes interrupts	-

5 LIBRARY DEPENDENCES

Table 5.1 shows the libraries used when instantiating the core (VHDL libraries).

Table 5.1: Library dependencies

Library	Package	Imported units	Description
IEEE	std_logic_1164	Types	Standard logic types
GRLIB	amba	Signals	AMBA signal definitions
GRLIB	config	Types	Amba P&P types
GRLIB	devices	Types	Device names and vendors
GRLIB	stdlib	All	Common VHDL functions
GAISLER	noelv	Signals	Counter vectors and types
BSC	pmu_module	Instances and signals	Instances and signal definitions for the SafeSU

6 INSTANTIATION

An example design is provided in the context of SELENE and De-RISC project. Integration examples of earlier releases of the unit along LEON3MP can be provided under demand.

Listing 1: SafeSU instance example for gpp_sys

```
-- Include BSC library
library bsc;
use bsc.pmu_module.all;

--Provide a non-overlaping hsidx
constant hsidx_pmu : integer := 6;

--Update the hsidx of the next slave
constant nextslv      : integer := hsidx_pmu + 1;

--Declare events  and signals of interest. They may change
--for each use case. Route such events to  pmu_events,
--ccs_contention and ccs_latency ports

--Instance of the unit

PMU_inst : ahb_wrapper
generic map(
ncpu   => CFG_NCPU,
hindex => hsidx_pmu,
haddr  => 16#801#,
hmask  => 16#FFF#
)
port map(
rst                => rstn,
clk                => clkm,
pmu_events         => pmu_events,
ccs_contention     => ccs_contention,
ccs_latency        => ccs_latency,
ahbsi              => ahbsi,
ahbso              => ahbso(hsidx_pmu));
```

Given the development status of the module and drivers it is recommended to not modify the internal parameters of the module.